File Handling

Introduction to File Handling **2**
Types of Files **3**
Text File **3**
Binary File **4**
FILE Structure **5**
fopen() Function **5**
File Opening Modes **6**
Writing Character to File **8**
Reading Character from File **9**
Writing String to File **11**
Reading String from File **12**
Formatted File Handling **18**
fprintf() Function **19**
fscanf() Function **24**
Binary File Handling **30**
fwrite() Function **30**
fread() Function **31**
fclose() Function **32**
File Position Functions **33**
rewind() **33**
fseek() **34**
ftell() **34**
Error Handling Functions **34**
feof() 35
ferror() 35
perror() 35
remove() Function **35**
rename() Function **35**
Embedded Systems File Handling Applications **35**
SD Card Logging 36
Black Box Systems 36
Firmware Update Systems 36
Common File Handling Errors **36**
Practice Questions **36**
MCQs **37**

TechnoScript

Introduction to File Handling

File handling in C is used to store data permanently inside storage devices. Normally variables store data temporarily in RAM.

When program execution ends:

- RAM data gets erased
- Variable values are lost

To store data permanently we use files.

Real Life Example

Suppose:

- Student marks stored in variable
- System turns OFF
- Data lost

To avoid this problem store data in file. Files store data permanently.

Why File Handling is Important

File handling is used for:

- Permanent data storage
- Data logging
- Configuration storage
- Sensor data storage
- Report generation
- Communication logs
- Database creation
- Firmware update storage

File Handling in Embedded Systems

In embedded systems file handling is used in:

- SD Card interfacing
- EEPROM storage
- Flash memory logging

TechnoScript

- Sensor data recording
- Black box systems
- Event logging
- Data acquisition systems

What is a File?

A file is a collection of data stored permanently in storage device.

Examples:

- .txt
- .bin
- .csv
- .dat
- .log

Types of Files

There are two modes of files:

1. Text File
2. Binary File

Text File

Text files store data in human readable format.

Example:

100
Hello
45.6

Characteristics of Text File

- Human readable
- Easy to edit
- More storage required
- Slower processing

TechnoScript

Examples

- .txt
- .csv
- .log

Binary File

Binary files store data in binary format. Data is stored exactly as present in memory.

Characteristics of Binary File

- Not human readable
- Faster access
- Less storage
- Efficient for embedded systems

Example

Suppose integer: 100
Stored in binary file as: 01100100

Difference Between Text File and Binary File

Text File	Binary File
Human readable	Not human readable
Slower	Faster
More storage space	Less storage space
Easy to debug	Difficult to debug
Uses characters	Uses raw binary

File Pointer

Definition

A file pointer is a pointer used to access and manage files.

TechnoScript

C uses file pointer to perform:

- Reading
- Writing
- Appending
- Updating

Syntax of File Pointer
FILE *fp;

FILE is predefined structure available in stdio.h hearder file.

FILE Structure

FILE is a structure defined inside standard library.

It contains information about:

- Current file position
- File mode
- Buffer information
- Error indicators
- EOF status

Internally compiler creates this structure. Programmer uses pointer to access it.

Creating File Pointer
FILE *fp;

Here:

- fp is file pointer
- FILE is predefined structure datatype

fopen() Function

The fopen() function is used to **open an existing file or create a new file**.

Syntax

TechnoScript

FILE *fopen(const char *filename, const char *mode);

Example
FILE *fp;

fp = fopen("data.txt","r");

"data.txt" → File name.
"r" → Opening mode.

Return Value : If file opens successfully it returns file pointer. If file opening failsit returns NULL.

Important Program

```c
#include<stdio.h>

int main()
{
   FILE *fp;

   fp = fopen("data.txt","r");

   if(fp == NULL)
   {
      printf("File opening failed");
   }
   else
   {
      printf("File opened successfully");
   }

   return 0;
}
```

File Opening Modes

Mode	Meaning

r	Read
w	Write
a	Append
r+	Read + Write
w+	Read + Write
a+	Read + Append
rb	Read binary
wb	Write binary
ab	Append binary
rb+	Read + Write binary
wb+	Read + Write binary
ab+	Append + Read binary

Understanding Modes in Detail

r Mode

fopen("data.txt","r")

- Opens existing file for reading
- File must exist
- Pointer at beginning

w Mode

fopen("data.txt","w")

- Opens file for writing
- Creates new file if not present
- Deletes old data

a Mode

TechnoScript

fopen("data.txt","a")

- Opens file for appending
- Data added at end
- Old data preserved

r+ Mode

fopen("data.txt","r+")

- Read and write
- File must exist

w+ Mode

fopen("data.txt","w+")

- Read and write
- Old data erased

a+ Mode

fopen("data.txt","a+")

- Read and append
- Data written at end

Writing Character to File

fputc()

Definition

Writes single character to file.

Syntax

fputc(character, file_pointer);

TechnoScript

Example

```c
#include <stdio.h>

int main()
{
    FILE *fp;

    // Open the file in write mode
    // If the file does not exist, it will be created
    fp = fopen("data.txt", "w");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("Error: Unable to open the file.\n");
        return 1;   // Exit the program with an error
    }

    // Write character 'A' into the file
    fputc('A', fp);

    // Close the file after completing the operation
    fclose(fp);

    return 0;       // Successful program termination
}
```

Reading Character from File

fgetc()

Reads single character from file.

Syntax

fgetc(file_pointer);

Returns :

- Reads one character from the file.
- Returns the character read.
- Returns EOF (End Of File) when no more characters are available.

Example

Example 1:

```c
#include <stdio.h>

int main()
{
    FILE *fp;
    char ch;

    // Open the file in read mode
    fp = fopen("data.txt", "r");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("Error: Unable to open the file.\n");
        return 1;
    }

    // Read one character from the file
    ch = fgetc(fp);

    // Display the character read from the file
    printf("%c", ch);

    // Close the file
    fclose(fp);

    return 0;
}
```

Example 2:

Writing String to File

fputs()

Writes string to file.

Syntax

fputs(string, file_pointer);

Example

Example 1

```c
#include <stdio.h>

int main()
{
    FILE *fp;

    // Open the file in write mode
    // If the file does not exist, it will be created
    fp = fopen("data.txt", "w");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Write a string into the file
    fputs("Embedded Systems", fp);

    // Close the file after writing
    fclose(fp);

    return 0;
}
```

Example 2

```c
#include <stdio.h>

int main()
{
    FILE *fp;

    // Open the file in write mode
    // If the file does not exist, it will be created
    fp = fopen("data.txt", "w");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Write the string "Embedded Systems" into the file
    fputs("Embedded Systems", fp);

    // Close the file
    fclose(fp);

    return 0;
}
```

Reading String from File

fgets()

Reads string from file.

Syntax

fgets(buffer,size,file_pointer);

Example

```c
#include <stdio.h>

int main()
{
    FILE *fp;
    char str[50];

    // Open the file in read mode
    fp = fopen("data.txt", "r");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Read a string from the file
    // Maximum 49 characters are read
    // The last character is reserved for '\0'
    fgets(str, 50, fp);

    // Display the string read from the file
    printf("%s", str);

    // Close the file
    fclose(fp);

    return 0;
}
```

fgets() is used to **read a string (a line of text)** from a file or standard input.

Syntax

char *fgets(char *buffer, int size, FILE *file_pointer);

Arguments

1. buffer

char buffer[100];

- This is the array where the read data will be stored.
- It must be a character array (string buffer).
- fgets() writes the characters it reads into this array.

Example:

```
char buffer[20];
fgets(buffer, 20, fp);
```

The characters read from the file will be stored in buffer.

2. size
fgets(buffer, 20, fp);

Here 20 is the size.

- Maximum number of characters to read.
- fgets() reads at most size - 1 characters.
- One byte is reserved for the null character ('\0').

Example:

```
char buffer[10];
fgets(buffer, 10, fp);
```

Maximum characters stored:

9 characters + '\0'

3. file_pointer
FILE *fp;

- Points to the file from which data is read.
- Obtained using fopen().

Example:

```
fp = fopen("data.txt", "r");
fgets(buffer, 20, fp);
```

Reads from data.txt.

Return Value

TechnoScript

The `fgets()` function returns a **pointer to the character array (`buffer`)** if the data is read successfully. If `fgets()` reaches the **end of the file (EOF)** before reading any characters, or if a **reading error occurs**, it returns `NULL`.

char *result = fgets(buffer, 20, fp);

- **Success:** `fgets()` returns the address of `buffer`.
- **Failure or EOF:** `fgets()` returns `NULL`.

Therefore, we can check the return value to determine whether the data was read successfully:

```
if (fgets(buffer, 20, fp) != NULL)
{
    printf("Data read successfully");
}
```

If `fgets()` returns `NULL`, it indicates that **either the end of the file has been reached or an error occurred while reading**:

```
if (fgets(buffer, 20, fp) == NULL)
{
    printf("End of file or error");
}
```

How Many Characters Does It Read?

fgets() stops when **any one** of the following occurs:

Condition 1: Newline found

File:

Hello World
Embedded

Code:

char buffer[50];
fgets(buffer, 50, fp);

Buffer contains:

Hello World\n

Notice that the newline character is also stored.

Condition 2: Size limit reached

File:

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Code:

```
char buffer[5];
fgets(buffer, 5, fp);
```

Buffer contains: ABCD

Memory:

Index	Value
0	A
1	B
2	C
3	D
4	\0

Only size-1 characters are read.

Condition 3: EOF reached

File:

ABC

Code:

```
char buffer[20];
fgets(buffer, 20, fp);
```

Buffer contains:

TechnoScript

ABC

Reading stops at end of file.

Example 1: Reading One Line

File (data.txt):

Embedded Systems
Linux
STM32

Program:

```c
#include <stdio.h>

int main()
{
    FILE *fp;
    char buffer[50];

    // Open the file in read mode
    fp = fopen("data.txt", "r");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Read a line/string from the file
    // sizeof(buffer) gives the size of the buffer (50 bytes)
    // fgets() reads at most 49 characters and adds '\0'
    fgets(buffer, sizeof(buffer), fp);

    // Display the string read from the file
    printf("%s", buffer);

    // Close the file
    fclose(fp);
```

```
    return 0;
}
```

Output:

Embedded Systems

Only the first line is read.

Example 2: Reading Entire File Line by Line

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char buffer[100];

    // Open the file in read mode
    fp = fopen("data.txt", "r");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Read the file line by line
    // fgets() returns NULL when the end of file is reached
    // or when a reading error occurs
    while (fgets(buffer, sizeof(buffer), fp) != NULL)
    {
        // Display the line read from the file
        printf("%s", buffer);
    }

    // Close the file
    fclose(fp);
```

```
    return 0;
}
```

Output:
Embedded Systems
Linux
STM32

Internal Working

Suppose file contains:

Hello\n

and

char buffer[20];

fgets(buffer,20,fp);

Memory after reading:

Index	Value
0	H
1	e
2	l
3	l
4	o
5	\n
6	\0

Notice:

- Newline \n is stored.
- Null character \0 is automatically added.

TechnoScript

Difference Between fgets() and fgetc()

Feature	fgetc()	fgets()
Reads	One character	One line/string
Return Type	int	char *
Speed	Slower for strings	Faster for strings
Stores \0	No	Yes
Typical Use	Character processing	String/line processing

Formatted File Handling

Formatted file handling uses:

- fprintf()
- fscanf()

Similar to:

- printf()
- scanf()

fprintf() Function

fprintf() is used to write **formatted output** to a file. It works similarly to printf(), but instead of printing to the screen, it writes to a file.

Syntax

int fprintf(FILE *stream, const char *format, ...);
Arguments

1. stream

FILE *fp;

- Pointer to the file where data will be written.
- Obtained using fopen().

TechnoScript

Example:

FILE *fp = fopen("data.txt", "w");

2. format

"%d %s %.2f"

- Format string that specifies how data should be written.
- Same format specifiers as printf().

Common format specifiers:

Specifier	Meaning
%d	Integer
%c	Character
%f	Float
%lf	Double
%s	String
%x	Hexadecimal
%u	Unsigned Integer

3. ... (Variable Arguments)

The values to be written.

Example:

fprintf(fp, "%d %s", 10, "STM32");

Here:

- %d ← 10
- %s ← "STM32"

Return Value

int result = fprintf(fp, "Hello");

TechnoScript

On Success

Returns the **number of characters written**.

Example:

int n = fprintf(fp, "Hello");

n becomes: 5

because 5 characters were written.

On Error

Returns: negative value

Typically: -1

Example:

```
if (fprintf(fp, "Hello") < 0)
{
   printf("Write failed\n");
}
```

Example 1: Writing a String

```
#include <stdio.h>

int main()
{
    FILE *fp;

    // Open the file in write mode
    // If the file does not exist, it will be created
    // If the file already exists, its previous contents will be deleted
    fp = fopen("data.txt", "w");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
```

TechnoScript

```
    }

    // Write formatted data into the file
    fprintf(fp, "Hello World");

    // Close the file
    fclose(fp);

    return 0;
}
```

File Content
Hello World

Example 2: Writing Multiple Data Types

```
#include <stdio.h>

int main()
{
    FILE *fp;

    // Open the file in write mode
    // If the file does not exist, it will be created
    // If the file already exists, its previous contents will be deleted
    fp = fopen("student.txt", "w");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Declare and initialize student details
    int roll = 101;
    float cgpa = 8.75;
    char name[] = "Rahul";
```

```
    // Write formatted student information into the file
    // %d  -> integer (roll number)
    // %s  -> string (name)
    // %.2f -> float with 2 digits after decimal point
    fprintf(fp,
            "Roll No = %d\n"
            "Name = %s\n"
            "CGPA = %.2f\n",
            roll,
            name,
            cgpa);

    // Close the file
    fclose(fp);

    return 0;
}
```

File Content
Roll No = 101
Name = Rahul
CGPA = 8.75

Writing in a Loop

```
#include <stdio.h>

int main()
{
    FILE *fp;

    // Open the file in write mode
    // If the file does not exist, it will be created
    // If the file already exists, its previous contents will be
overwritten
    fp = fopen("table.txt", "w");

    // Check whether the file was opened successfully
```

```
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Write numbers from 1 to 10 into the file
    for (int i = 1; i <= 10; i++)
    {
        // Write the value of i followed by a newline
        fprintf(fp, "%d\n", i);
    }

    // Close the file after completing the writing operation
    fclose(fp);

    return 0;
}
```

File Content
1
2
3
4
5
6
7
8
9
10

Internal Working

Suppose:

int age = 25;

fprintf(fp, "Age = %d", age);

Internally:

TechnoScript

1. fprintf() sees %d.
2. Converts 25 into characters '2' and '5'.
3. Writes:

Age = 25

Difference Between fprintf() and printf()

Function	Output Destination
printf()	Screen (stdout)
fprintf()	File or any stream

Example
printf("Hello");

Output:

Hello

on the terminal.

fprintf(fp, "Hello");

Output: Hello → inside the file.

Difference Between fprintf() and fputs()

Feature	fprintf()	fputs()
Formatted Output	Yes	No
%d, %f, %s support	Yes	No
Writes String	Yes	Yes
Return Type	int	int

TechnoScript

`fscanf()` Function

`fscanf()` is used to **read formatted data from a file**.

It works similarly to `scanf()`, but instead of reading data from the **keyboard (`stdin`)**, `fscanf()` reads data from a **file stream**.

For example, it can read:

- Integer → `%d`
- Float → `%f`
- Character → `%c`
- String → `%s`

Function Prototype

int fscanf(FILE *stream, const char *format, ...);

Found in:

#include <stdio.h>

Arguments

1. `stream`

FILE *fp;

`stream` is the **file pointer** from which the data will be read.

The file pointer is obtained using `fopen()`:

fp = fopen("data.txt", "r");

Example:

fscanf(fp, "%d", &num);

Here, `fp` specifies that the data should be read from `data.txt`.

2. `format`

TechnoScript

The format string specifies **what type of data should be read**.

Common format specifiers:

Format	Data Type	Example
`%d`	`int`	`101`
`%f`	`float`	`8.75`
`%c`	`char`	`A`
`%s`	String	`Rahul`
`%lf`	`double`	`8.7567`

Example:

int roll;

fscanf(fp, "%d", &roll);

If the file contains: 101

Then: roll = 101

3. Additional Arguments

The variables where the data will be stored are passed as arguments. For most data types, we pass the **address of the variable** using `&`.

Example:

int age;

fscanf(fp, "%d", &age);

Here:

%d → tells fscanf() to read an integer
&age → tells fscanf() where to store the integer

For a character array (string), we normally don't use `&`:

TechnoScript

char name[50];

fscanf(fp, "%s", name);

Example

Suppose `data.txt` contains:

101 Rahul 8.75

Program:

```c
#include <stdio.h>

int main()
{
    FILE *fp;

    int roll;
    char name[50];
    float cgpa;

    // Open the file in read mode
    fp = fopen("data.txt", "r");

    // Check whether the file was opened successfully
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Read roll number, name and CGPA from the file
    fscanf(fp, "%d %s %f", &roll, name, &cgpa);

    // Display the data
    printf("Roll No = %d\n", roll);
    printf("Name = %s\n", name);
    printf("CGPA = %.2f\n", cgpa);

    // Close the file
    fclose(fp);
```

```
    return 0;
}
```

Output

Roll No = 101
Name = Rahul
CGPA = 8.75

Return Value

`fscanf()` returns the **number of input items successfully matched and assigned**.

For example:

int result;

result = fscanf(fp, "%d %s %f", &roll, name, &cgpa);

If all three values are successfully read: result = 3

If only two values are successfully read: result = 2

If no input item is successfully matched: result = 0

If **end-of-file is encountered before any item is successfully read**, it returns: EOF

Important

Unlike `fgets()`, `fscanf()` does **not return** `NULL`.

Return Value Example

int result;

result = fscanf(fp, "%d %s %f", &roll, name, &cgpa);

if (result == 3)
{
   printf("All data read successfully\n");
}
else
{

TechnoScript

```
    printf("Data reading failed\n");
}
```

`fscanf()` with Different Data Types

Suppose `data.txt` contains:

101 Rahul 8.75 A

Program:

```c
#include <stdio.h>

int main()
{
    FILE *fp;

    int roll;
    char name[50];
    float cgpa;
    char grade;

    // Open file in read mode
    fp = fopen("data.txt", "r");

    // Check file opening
    if (fp == NULL)
    {
        printf("File opening failed\n");
        return 1;
    }

    // Read formatted data from the file
    fscanf(fp, "%d %s %f %c",
           &roll,
           name,
           &cgpa,
           &grade);

    // Display the data
    printf("Roll  = %d\n", roll);
```

```
    printf("Name  = %s\n", name);
    printf("CGPA  = %.2f\n", cgpa);
    printf("Grade = %c\n", grade);

    // Close file
    fclose(fp);

    return 0;
}
```

Output:

Roll = 101
Name = Rahul
CGPA = 8.75
Grade = A

Reading Multiple Records Using `fscanf()`

Suppose `student.txt` contains:

101 Rahul 8.75
102 Amit 7.90
103 Priya 9.10

We can read all records using a loop.

```
#include <stdio.h>

int main()
{
    FILE *fp;

    int roll;
    char name[50];
    float cgpa;

    // Open the file in read mode
    fp = fopen("student.txt", "r");

    // Check whether the file was opened successfully
    if (fp == NULL)
```

```
    {
        printf("File opening failed\n");
        return 1;
    }

    // Read records until fscanf() cannot read all three values
    while (fscanf(fp, "%d %s %f", &roll, name, &cgpa) == 3)
    {
        printf("Roll = %d, Name = %s, CGPA = %.2f\n",
               roll, name, cgpa);
    }

    // Close the file
    fclose(fp);

    return 0;
}
```

Output

Roll = 101, Name = Rahul, CGPA = 8.75
Roll = 102, Name = Amit, CGPA = 7.90
Roll = 103, Name = Priya, CGPA = 9.10

This is one of the most common patterns for `fscanf()`:

```
while (fscanf(fp, "%d %s %f", &roll, name, &cgpa) == 3)
{
   // process data
}
```

Important Difference: `fgets()` vs `fscanf()`

Feature	`fgets()`	`fscanf()`
Purpose	Reads a line/string	Reads formatted data
Input source	File/`stdin`	File/stream
Reads integer directly	No	Yes
Reads float directly	No	Yes

Reads character	As part of string	Yes
Reads string	Yes	Yes
Reads spaces in string	Yes	No, with `%s`
Return type	`char *`	`int`
Success return	Pointer to buffer	Number of items read
Failure/EOF	`NULL`	`0` or `EOF`

fgets() vs fscanf() Example

Suppose the file contains:

101 Rahul 8.75

Using `fgets()`

char buffer[100];

fgets(buffer, sizeof(buffer), fp);

The entire line is stored as a **string**: "101 Rahul 8.75\n\0"

You would need additional processing to extract `101`, `Rahul`, and `8.75`.

Using `fscanf()`

fscanf(fp, "%d %s %f", &roll, name, &cgpa);

The data is directly separated into variables:

roll = 101
name = "Rahul"
cgpa = 8.75

Binary File Handling

Binary file handling uses:

TechnoScript

- fwrite()
- fread()

fwrite() Function

Definition

Writes binary data into file.

Syntax

```
fwrite(data,
       size,
       count,
       file_pointer);
```

Example

```
#include <stdio.h>

struct Student
{
   int roll;
   float marks;
};

int main()
{
   FILE *fp;

   // Create and initialize a structure variable
   struct Student s1 = {1, 95.5};

   // Open the file in binary write mode
   // "wb" means write + binary mode
   fp = fopen("student.dat", "wb");

   // Check whether the file was opened successfully
   if (fp == NULL)
   {
      printf("File opening failed\n");
      return 1;
```

TechnoScript

```
    }

    // Write the structure data into the binary file
    //
    // &s1       -> address of the structure
    // sizeof(s1)-> size of one structure
    // 1         -> number of structures to write
    // fp        -> file pointer
    fwrite(&s1, sizeof(s1), 1, fp);

    // Close the file
    fclose(fp);

    return 0;
}
```

fread() Function

Definition

Reads binary data from file.

Syntax

```
fread(data,
      size,
      count,
      file_pointer);
```

Example

```
#include<stdio.h>

struct Student
{
    int roll;
    float marks;
};

int main()
{
    FILE *fp;
```

TechnoScript

```
    struct Student s1;

    fp = fopen("student.dat","rb");

    fread(&s1,
          sizeof(s1),
          1,
          fp);

    printf("%d %.2f",
           s1.roll,
           s1.marks);

    fclose(fp);

    return 0;
}
```

fclose() Function

Definition

Closes file.

Syntax

fclose(file_pointer);

Importance

- Saves data properly
- Releases resources
- Prevents corruption

EOF in File Handling

EOF means End Of File.

When file ends: EOF returned

Technos

Example

```
while((ch=fgetc(fp)) != EOF)
{
    printf("%c", ch);
}
```

File Position Functions

These functions control file pointer position.

Functions:

- rewind()
- fseek()
- ftell()

rewind()

Moves pointer to beginning.\

Syntax

rewind(fp);

fseek()

Moves pointer to specific location.

Syntax

fseek(fp,
    offset,
    position);

Positions

Position	Meaning

TechnoScript

SEEK_SET	Beginning
SEEK_CUR	Current
SEEK_END	End

Example
fseek(fp,5,SEEK_SET);

ftell()

Returns current file position.

Syntax
ftell(fp);

Example
printf("%ld", ftell(fp));

Error Handling Functions

feof()

Checks EOF.

ferror()

Checks file error.

perror()

Prints error message.

Example
perror("File Error");

TechnoScript

remove() Function

Deletes file.

Syntax
remove("data.txt");

rename() Function

Renames file.

Syntax
rename("old.txt",
"new.txt");

Embedded Systems File Handling Applications

SD Card Logging

Temperature logs
Pressure logs
Vehicle tracking

Black Box Systems

Stores:

- Crash data
- Sensor values
- Error events

Firmware Update Systems

TechnoScript

Binary files store firmware images.

Common File Handling Errors

1. Forgetting fclose()
2. Using wrong mode
3. Not checking NULL
4. Reading unopened file
5. Buffer overflow
6. Using invalid pointer

Practice Questions

Basic Questions

1. What is file handling?
2. What is file pointer?
3. Difference between text and binary file?
4. What is fopen()?
5. Why fclose() important?
6. Difference between fprintf and fwrite?
7. Difference between fscanf and fread?
8. What is EOF?
9. Explain file opening modes.
10. Why binary files faster?
11. Explain FILE structure.
12. Difference between text and binary storage.
13. Explain fseek().
14. Explain rewind().
15. Explain ftell().
16. What happens if fclose() not called?
17. Explain buffering.
18. Why binary mode preferred in embedded systems?
19. Explain SD card logging.
20. Difference between append and write mode.

Practice Programs

1. Create file and write character.
2. Read character from file.

TechnoScript

3. Write string using fputs().
4. Read string using fgets().
5. Copy one file to another.
6. Count characters in file.
7. Count lines in file.
8. Store structure in binary file.
9. Read structure from binary file.
10. Create sensor logging system.

MCQs

Q1.

Which function writes single character?

A) fputs()
B) fprintf()
C) fputc()
D) fwrite()

Answer: C

Q2.

Which mode preserves old data?

A) w
B) r
C) a
D) wb

Answer: C

Q3.

Which function reads binary data?

A) fscanf()
B) fgets()
C) fread()
D) fgetc()

TechnoScript

Answer: C

Q4.

What fopen() returns on failure?

A) EOF
B) 0
C) NULL
D) -1

Answer: C

Q5.

Which function moves file pointer?

A) fopen()
B) fclose()
C) fseek()
D) fread()

Answer: C

TechnoScript